```
vectors = model.encode(documents).
tolist()

client.upsert(
    collection_name="docs",
    points=[
        PointStruct(id=i, vector=v,
payload={"text": documents[i]})
        for i, v in enumerate(vectors)
    ],
)

query = "Tell me about pet animals"
query_vector = model.encode(query).
tolist()

results = client.search(
    collection_name="docs",
    query_vector=query_vector,
    limit=3,
)

for r in results:
    print(f"Score: {r.score:.4f} -
{r.payload['text']}")
```

Run that and you'll see the search returns the cat sentence, the dog sentence, and the cats-and-dogs sentence as the top three. The Linux and Python sentences are further away. Nobody told the database about pets specifically. The embedding model encoded the meaning, and the database found the closest matches.

The first time you see this work, it feels a little like magic. It's not magic, of course. It's just a lot of maths behind a clean API.

Where pgvector fits in

If you're already running Postgres, pgvector is genuinely worth knowing about. It's an extension that adds a vector data type and vector similarity operators to your existing database.

```
Setting it up is straightforward.
CREATE EXTENSION vector;
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    content TEXT,
    embedding vector(384)
);
```

Insertion looks like regular Postgres.

```
INSERT INTO documents (content,
embedding) VALUES
    ('The cat sat on the mat.', '[0.1,
0.2, ...]');
```

Searching for similar vectors uses a new operator.

```
SELECT content
FROM documents
ORDER BY embedding <-> '[0.15, 0.21,
...]'
LIMIT 5;
```

That <-> operator computes the Euclidean distance between two vectors. There are also operators for cosine distance and inner product. You can build indexes on the vector column using HNSW or IVFFlat algorithms, which makes search fast even at millions of rows.

The big advantage of pgvector is that you can do vector search and traditional SQL queries in the same database and in the same query. Want to find documents similar to this query, but only from this customer, only created in the last month, and only tagged as urgent? One query. With most pure vector databases, this kind of compound filtering requires either denormalising your data or making multiple round trips.

For a lot of teams, pgvector is the right starting point. You only graduate to a dedicated vector database when you need the performance or features they offer.

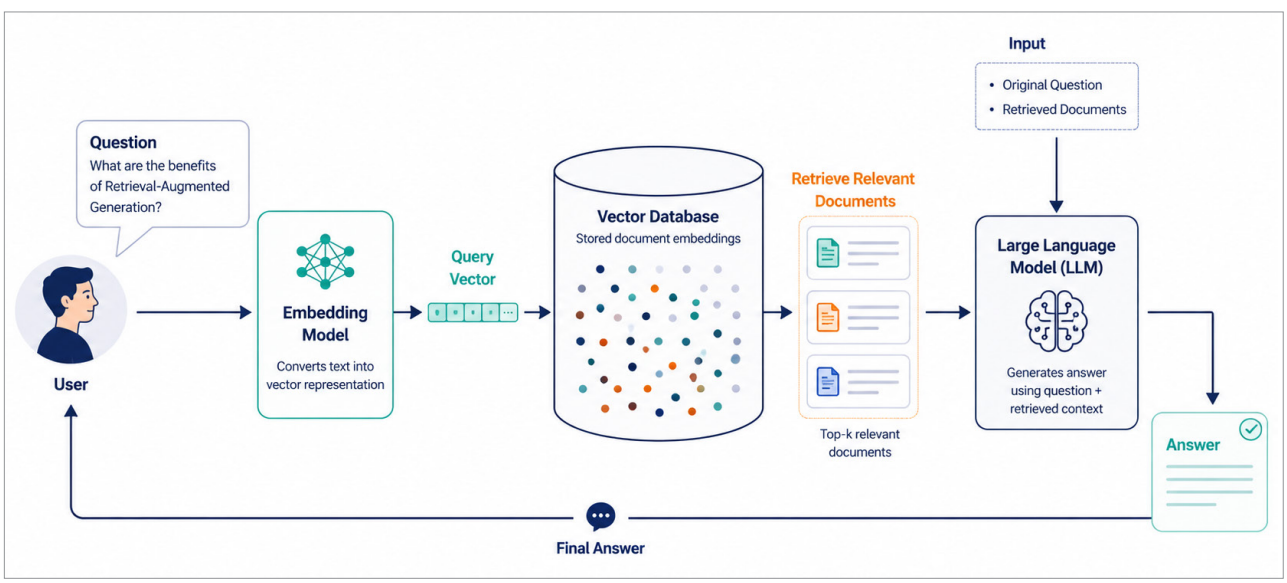

Figure 2: A typical RAG pipeline using a vector database